

FIFA

Create a folder and open it in VS code.

Create a virtual environment -> `python -m venv fifa_env`

(fifa_env is a virtual environment — basically a separate little Python setup just for this project)

Activate it -> `fifa_env\Scripts\activate`

Now run -> `pip install opencv-python librosa moviepy numpy pandas scikit-learn matplotlib yt-dlp`

This downloads the Python packages into your FIFA project's virtual environment.

Library	Why we'll use it
opencv-python	Read/process video frames
librosa	Audio processing
moviepy	Cut and combine video clips
numpy	Numerical calculations
pandas	Work with event/label data
scikit-learn	ML utilities + evaluation
matplotlib	Graphs/visualization
yt-dlp	Download videos from supported sites

And your project will eventually look something like this:

FIFA-Automatic-Highlights/

```
|
|
|└─ fifa_env/
|
|
|└─ data/
|
|  └─ videos/
|
|  └─ labels/
|
|    └─ processed/
|
|
|└─ src/
|
|  └─ preprocessing.py
|
|  └─ feature_extraction.py
|
|  └─ event_detection.py
|
|    └─ highlight_generation.py
|
|
|└─ models/
|
|
|└─ outputs/
|
|
└─ main.py
```

Downloading a full match video, that is Argentina vs France 2022 Final, with this command, -> yt-dlp -f "best[height<=480]" "VIDEO_URL" .

yt-dlp is a command-line tool that can download videos from websites that it supports.

Hiccup #1: YouTube mostly stopped offering combined (video+audio in one file) formats above 360p years ago. Anything 480p and up is almost always separate video-only and audio-only streams.

yt-dlp -F "<https://www.youtube.com/live/RgqKdplLlk4>" to see the list of formats

yt-dlp -f "135+140" "<https://www.youtube.com/live/RgqKdplLlk4>" the 135 is video and 140 is audio format. We are combining the video and audio together.

Hiccup solved

extract_audio.py code:

```
from moviepy import VideoFileClip

video = VideoFileClip("match.mp4")
video.audio.write_audiofile("match_audio.wav")
```

audio_energy.py code:

```
import librosa
import matplotlib.pyplot as plt

y, sr = librosa.load("match_audio.wav")
energy = librosa.feature.rms(y=y)[0]

plt.plot(energy)
plt.savefig("audio_energy.png")
```

After executing both orderly, u will get a PNG image in the folder with audio spikes.

So, the next step is observing the audio spikes, if the spikes were for an instant high, it shouldn't be considered as a highlight, Here's why that matters — a single loud second could be a stray shout, a mic bump, static, or one enthusiastic fan. A real highlight (goal, big chance) tends to produce a sustained roar lasting several seconds, building and then settling. So filtering for sustained spikes cuts out noise-y false positives.

Code for threshold_detector.py:

```
import librosa
import numpy as np

# ---- Load audio ----
y, sr = librosa.load("match_audio.wav")

# ---- Compute energy ----
energy = librosa.feature.rms(y=y)[0]

# ---- Convert frame indices to actual timestamps ----
times = librosa.frames_to_time(np.arange(len(energy)), sr=sr)

# ---- Set threshold: mean + 2*std ----
threshold = energy.mean() + 2 * energy.std()
highlight_frames = energy > threshold

# ---- Require sustained spikes (not just a single loud frame) ----
# min_duration_frames controls how many consecutive "loud" frames
# are needed before we trust it as a real highlight, not noise.
# Tune this number based on how long your energy array is vs match length.
min_duration_frames = 20

smoothed = np.convolve(highlight_frames, np.ones(min_duration_frames),
mode='same') >= min_duration_frames

# ---- Group consecutive True frames into (start_time, end_time) windows ----
highlight_windows = []
in_highlight = False
start_time = None

for i, is_hl in enumerate(smoothed):
    if is_hl and not in_highlight:
        # entering a highlight region
        in_highlight = True
        start_time = times[i]
    elif not is_hl and in_highlight:
        # leaving a highlight region
```

```

        in_highlight = False
        end_time = times[i]
        highlight_windows.append((start_time, end_time))

# catch case where match ends while still inside a highlight
if in_highlight:
    highlight_windows.append((start_time, times[-1]))

# ---- Print results ----
print(f"Found {len(highlight_windows)} candidate highlight windows:\n")
for start, end in highlight_windows:
    duration = end - start
    print(f"  {start:.1f}s -> {end:.1f}s (duration: {duration:.1f}s)")

```

Found 30 candidate highlight windows:

```

1356.6s -> 1356.9s (duration: 0.3s)
1440.4s -> 1441.4s (duration: 1.0s)
2217.3s -> 2217.3s (duration: 0.0s)
2218.1s -> 2220.0s (duration: 1.9s)
2221.3s -> 2221.4s (duration: 0.0s)
4099.0s -> 4099.0s (duration: 0.0s)
5316.4s -> 5316.5s (duration: 0.1s)
5457.9s -> 5458.7s (duration: 0.8s)
5463.1s -> 5463.2s (duration: 0.1s)
5483.5s -> 5483.5s (duration: 0.0s)
6396.5s -> 6396.6s (duration: 0.1s)
7475.9s -> 7475.9s (duration: 0.0s)
7546.6s -> 7546.7s (duration: 0.1s)
7790.8s -> 7791.0s (duration: 0.2s)
7792.3s -> 7793.0s (duration: 0.7s)
7793.5s -> 7793.8s (duration: 0.3s)

```

7797.6s -> 7797.7s (duration: 0.1s)
7816.8s -> 7817.0s (duration: 0.2s)
7901.4s -> 7901.4s (duration: 0.0s)
8355.2s -> 8355.4s (duration: 0.2s)
8496.3s -> 8496.6s (duration: 0.3s)
8637.5s -> 8637.7s (duration: 0.2s)
8678.2s -> 8678.3s (duration: 0.1s)
8685.3s -> 8685.3s (duration: 0.0s)
8933.4s -> 8933.6s (duration: 0.1s)
9086.7s -> 9086.7s (duration: 0.0s)
9090.4s -> 9090.5s (duration: 0.0s)
9187.2s -> 9187.4s (duration: 0.2s)
9189.1s -> 9189.2s (duration: 0.1s)
9191.3s -> 9191.5s (duration: 0.2s)

All are milliseconds duration, so not our desired outcome.

By tweaking after sometime, we got

Total frames: 405689

Frames above threshold: 49027

Threshold value: 0.0477

Energy min/mean/max: 0.0007 / 0.0218 / 0.0839

frame_duration: 0.0232s, min_duration_frames: 43

Frames marked highlight after smoothing: 24524

Raw candidate windows: 292

After merging: 215 windows

After filtering (≥ 5.0 s): 33 windows

Top 20 highlight candidates (by intensity):

1346.5s -> 1353.7s (duration: 7.2s, peak energy: 0.0766)
1355.8s -> 1367.7s (duration: 11.9s, peak energy: 0.0766)
2212.6s -> 2228.1s (duration: 15.5s, peak energy: 0.0771)
5288.2s -> 5294.6s (duration: 6.5s, peak energy: 0.0772)
5373.6s -> 5385.9s (duration: 12.3s, peak energy: 0.0771)
5455.8s -> 5470.2s (duration: 14.4s, peak energy: 0.0835)
5478.9s -> 5499.3s (duration: 20.4s, peak energy: 0.0793)
5502.5s -> 5513.4s (duration: 10.9s, peak energy: 0.0787)
6300.5s -> 6309.9s (duration: 9.4s, peak energy: 0.0792)
7471.6s -> 7478.7s (duration: 7.0s, peak energy: 0.0793)
7788.7s -> 7809.8s (duration: 21.1s, peak energy: 0.0802)
7849.0s -> 7854.8s (duration: 5.8s, peak energy: 0.0764)
7881.0s -> 7886.1s (duration: 5.1s, peak energy: 0.0796)
7889.1s -> 7901.8s (duration: 12.7s, peak energy: 0.0777)
7925.2s -> 7933.0s (duration: 7.9s, peak energy: 0.0795)
8365.6s -> 8371.7s (duration: 6.1s, peak energy: 0.0772)
8676.2s -> 8695.4s (duration: 19.2s, peak energy: 0.0767)
9037.6s -> 9045.3s (duration: 7.7s, peak energy: 0.0772)
9085.1s -> 9092.0s (duration: 6.8s, peak energy: 0.0774)
9186.4s -> 9197.7s (duration: 11.3s, peak energy: 0.0784)

If these are wrong at the end of the project we know where we should change

Code for generate_highlights.py:

```

from moviepy import VideoFileClip, concatenate_videoclips

# ---- Paste your top windows here (from threshold_detector.py output) ----
highlight_windows = [
    (1346.5, 1353.7),
    (1355.8, 1367.7),
    (2212.6, 2228.1),
    (5288.2, 5294.6),
    (5373.6, 5385.9),
    (5455.8, 5470.2),
    (5478.9, 5499.3),
    (5502.5, 5513.4),
    (6300.5, 6309.9),
    (7471.6, 7478.7),
    (7788.7, 7809.8),
    (7849.0, 7854.8),
    (7881.0, 7886.1),
    (7889.1, 7901.8),
    (7925.2, 7933.0),
    (8365.6, 8371.7),
    (8676.2, 8695.4),
    (9037.6, 9045.3),
    (9085.1, 9092.0),
    (9186.4, 9197.7),
]

# ---- Add a small buffer before/after each clip for context ----
buffer_before = 1.5 # seconds before the detected spike
buffer_after = 1.5  # seconds after the detected spike

print("Loading video...")
video = VideoFileClip("match.mp4")
video_duration = video.duration

clips = []
for i, (start, end) in enumerate(highlight_windows):
    padded_start = max(0, start - buffer_before)
    padded_end = min(video_duration, end + buffer_after)
    print(f"Cutting clip {i+1}/{len(highlight_windows)}: {padded_start:.1f}s -> {padded_end:.1f}s")
    clip = video.subclipped(padded_start, padded_end)
    clips.append(clip)

```



```
print("Concatenating clips...")
final = concatenate_videoclips(clips)

print("Writing final highlight reel...")
final.write_videofile("highlight_reel.mp4", codec="libx264", audio_codec="aac")

print("Done! Saved as highlight_reel.mp4")
```

AND FINALLY!! You will get the highlight_reel.mp4 in your folder. DONE!!

Mine came out PERFECT! YAYYY!!!

Thank you Claude and ChatGPT.